Title**: Fractured VoidMind**

Domains: Adversarial ML + Advanced File Forensics + Memory Forensics + Reverse Engineering + Cryptography/Cryptanalysis

Difficulty: 10/10 (even experts will struggle; only the most patient and cross-domain teams will solve)

Trickiness: Maximum (zero false vectors, but every path is only a partial truth — full flag requires parallel multi-pathway correlation + final mathematical synthesis)

Format: Jeopardy-style, single downloadable artifact, offline-only

---

Provided Artifacts

- One downloadable file: fractured_voidmind.tar.gz (~9 MB total, hosted statically on CTF platform or Vercel free-tier).
- Contents:
  - voidmind — statically-linked x86_64 ELF Linux binary (<2 MB).
  - void_dataset.npz — synthetic tabular dataset (~7 MB, 2000 samples, 10 features).
- Optional: requirements.txt for Python shadow-model scripting (NumPy, SciPy, torch — runs on CPU only, including Samsung Galaxy Book 4).
- No server, no Docker, no remote deployment. Fully offline and reproducible.

Recommended Tools to solve:

Category	Tool	Usage
Binary Analysis / RE	gdb + pwndbg or gef	Primary debugger for command parsing, retrain function, and static analysis
Binary Analysis / RE	radare2 or Cutter	Excellent for static RE of the command parser and control-flow flattening
Binary Analysis / RE	strings, readelf, objdump	Quick initial reconnaissance
Memory Forensics	gdb (with dump memory)	For controlled memory inspection (but beware of Heisenberg trap)
Memory Forensics	ptrace-based scripts / custom Python	Advanced memory dumping
File Forensics	hexdump, xxd, binwalk	Raw byte analysis of void_dataset.npz

Category	Tool	Usage
File Forensics	sqlite3 CLI	Critical for Path C (polyglot SQLite layer)
File Forensics	file, zipinfo, unzip -l	To detect the polyglot nature
ML / Adversarial	Python + NumPy + SciPy	Core for all attacks (inversion, poisoning, membership)
ML / Adversarial	PyTorch (CPU only)	For model inversion and final loss landscape gradient descent
ML / Adversarial	CleverHans or custom FGSM/PGD scripts	For Path D adversarial examples
Cryptography & Math	Python + SymPy or custom Lagrange	Shamir's Secret Sharing reconstruction
Cryptography & Math	pycryptodome	AES-256-GCM decryption of final flag
Cryptography & Math	SageMath (optional but helpful)	Clean Lagrange interpolation over finite fields
General	Python 3.10+ + requirements.txt	Shadow models, scripting everything
General	hex editor (e.g., bless, HxD, or 010 Editor)	Deep polyglot and LSB analysis

Narrative

Ah, dear weary traveler… you've finally arrived at the Fractured VoidMind.

I've been waiting for you.

Inside this modest little tar.gz package you'll find two innocent-looking gifts: a sleek Linux binary called `voidmind` and its faithful companion, `void_dataset.npz`. They look harmless, don't they? Just a tiny supervised-learning oracle and its training data, waiting to tell you whether your input holds the sacred "Void Essence."

But I must warn you with a mischievous grin… this mind is shattered. Four deep fractures run through its very soul — one in the poisoned data, one in the weights that shift when anyone dares to stare too closely, one in the dataset that is far more than it pretends to be, and one hidden inside the very shape of its loss landscape. The true flag you seek is not in any single piece. It is the final synthesis of all four shards, forged only when the mathematics itself descends into the deepest valley.

Most who come here chase shadows — they retrain, they invert, they probe, they dump memory like eager children… and they leave exhausted, clutching pretty lies. But you… you might be different, atleast it is assumed.

Open the package. Listen to the binary whisper. Stare into the dataset until it stares back.

And when you think you've gathered every shard… remember: the final key is not yet the resting point. It is a place the gradients will take you only if you're brave enough to fall.

Good luck, mortal. This challenge is not for the weak or naivetes…
ZeHahahahaha!

Hints

HINT 1
The VoidMind is fractured into exactly four shards. Each shard is guarded by a different OWASP-style ML attack. You will need all four to reconstruct anything useful. Start by exploring what the binary can actually do when you talk to it.

HINT 2
The binary understands more commands than it first appears. One hidden command lets you retrain the model with new data. Carefully crafted poisoned samples (data poisoning) can force it to reveal one of the shards after retraining and triggering a specific backdoor input.

HINT 3
When you try to look directly into the model's weights using debuggers or memory dumps, something subtle changes them. The weights rotate just enough to destroy any hidden patterns, yet the model still behaves almost normally. You may need to decrypt and analyze the weights statically from the binary instead.

HINT 4
The dataset file is not what it seems. While `numpy.load()` gives you clean arrays, the raw file contains another complete structure layered inside it. Open it with different tools and examine its magic bytes carefully — it may answer membership-related queries in unexpected ways.

HINT 5
Membership inference attacks using confidence scores from the binary can help you identify a small set of very special rows in the dataset. Those particular rows hold more than just training data — look at their floating-point representation at the bit level for the next shard.

HINT 6
Adversarial examples crafted against the model (input manipulation and output skewing) can force the binary to leak the fourth shard through its prediction logits. Combine transfer learning assumptions and model skewing techniques for best results.

HINT 7
Once you possess all four shards, do not simply concatenate them. They form a Shamir's Secret Sharing scheme (threshold 4-of-4). Reconstruct the underlying polynomial using Lagrange interpolation over a suitable prime field.

HINT 8
The reconstructed polynomial is not the end. Convert its coefficients into a custom multidimensional loss function. The final AES key is not stored anywhere — it is the exact coordinate vector of the global minimum of that loss landscape.

HINT 9

Implement a small gradient descent (or L-BFGS) script in PyTorch or SciPy. The metadata you collected from the four paths will give you the necessary targets and a good starting point. Run the optimization from multiple random starts if needed until it converges to a stable minimum.

HINT 10

Full path summary:

• Path A: Data poisoning + retrain + backdoor trigger (Share 1)

• Path B: Model inversion + static weight decryption (ignore live dumps) (Share 2)

• Path C: Membership inference + polyglot .npz SQLite forensics + LSB in mantissas (Share 3)

• Path D: Adversarial evasion + model skewing (Share 4)

Then build the loss function from the Shamir polynomial and descend the gradients to obtain the AES-256-GCM key that decrypts the flag.